

LLM Inference Optimization

Benchmarking Techniques for Headline Generation

Final Project Report

Executive Summary

This report benchmarks a set of inference optimization techniques applied to a transformer-based headline generation task. The target model for the core comparison is Llama-3.2-3B, evaluated on a news headline dataset. Each technique is assessed along three axes: latency (time to generate a headline), throughput (tokens produced per second), and output quality (ROUGE overlap against reference headlines).

The headline finding is clear: 4-bit quantization delivers the largest performance improvement among directly comparable configurations, reducing mean latency from approximately 18 seconds to 0.59 seconds (about a 30x speedup) while cutting memory use by roughly 69%, with only a modest impact on output quality. It is the recommended strategy for production deployment. KV caching provides the essential first-order speedup at no quality cost. Distributed-inference techniques (pipeline/tensor parallelism and DeepSpeed) were investigated within a single-GPU constraint and are documented as enabling-technologies for larger models rather than latency wins for this 3B workload.

Methodology

Task and Data

The task is abstractive headline generation: given the body text of a news article, the model produces a short headline. Quality is measured with ROUGE (Recall-Oriented Understudy for Gisting Evaluation), which scores n-gram overlap between the generated headline and a human-written reference. ROUGE-1 measures single-word overlap, ROUGE-2 measures consecutive word-pair overlap, and ROUGE-L measures the longest common subsequence.

Models

The core optimizations (baseline, KV caching, pruning, quantization) were applied to Llama-3.2-3B. Speculative decoding was implemented separately using a gpt2-medium target model with a gpt2 draft model. Because these are different model families, the speculative decoding row is reported but is not directly comparable to the Llama-based rows on either latency or quality.

Libraries and Tools

- PyTorch and Hugging Face Transformers for model loading and generation.
- bitsandbytes for 4-bit quantization (NF4).
- torch.nn.utils.prune for L1 unstructured weight pruning.
- rouge_score for quality evaluation (used directly rather than via the evaluate library, which had a version conflict in the environment).
- torch.profiler for operator-level GPU profiling.

Measurement Protocol

All timing was captured on a single NVIDIA Tesla T4 GPU. A warm-up generation run precedes every measured run to exclude one-time CUDA initialization cost, and `torch.cuda.synchronize()` is called before and after timing to account for asynchronous GPU execution. Throughput is computed as total tokens generated divided by total elapsed time. ROUGE is averaged over a fixed sample of 25 articles using an identical prompt across all techniques, so that quality differences reflect the technique rather than single-sample noise.

Benchmark Results

The table below summarizes performance, memory, and quality for each technique over a fixed sample of 25 articles (N=1 for the parallelism simulations, which measure a single representative run). The four shaded rows are advanced or distributed-inference stages whose results carry the caveats noted below; the upper four rows are directly comparable Llama-3.2-3B configurations.

Technique	Latency (s)	Throughput	ROUGE-1	Memory (MB)	Notes	N
Baseline (No Cache)	17.96	0.95	0.191	4714	reference	25
KV Caching	4.52	3.58	0.156	4714	lossless*	25
Pruning (30%)	4.78	3.76	0.182	4714	no dense gain	25
Quantization (4-bit)	0.59	29.97	0.142	1466	best overall	25
Pipeline Parallel (sim)†	1.59	31.44	—	769	per-device	1
Tensor Parallel (sim)†	1.53	32.76	—	2472	per-device	1
Spec. Decoding (K=3)‡	1.29	39.42	0.093‡	914	diff. model	25
DeepSpeed Inference§	1.47	34.07	—	2472	int8 applied	25

* KV caching is mathematically lossless in principle. The small ROUGE difference versus baseline reflects floating-point or sampling non-determinism between the cached and uncached code paths; quality is effectively unchanged.

† Pipeline- and tensor-parallel results are single-GPU simulations of the layer-partition (PP) and weight-shard (TP) mechanics; the memory figure is per-device (total ÷ 2). On one T4 these cannot realize true multi-GPU speedup, so latency reflects the underlying model, not a parallelism gain. The TP and PP rows were measured on different underlying models (fp16 vs 4-bit), which is why their per-device memory differs.

‡ Speculative decoding uses gpt2-medium / gpt2, not Llama-3.2-3B. Its ROUGE (0.093) is low because base gpt2 is not instruction-tuned and continues the article rather than producing a headline — a property of the model, not the technique (which is lossless relative to its own target). Latency and ROUGE are not comparable to the Llama rows.

§ DeepSpeed Inference (v0.17.4) ran on the fp16 model with tensor_parallel tp_size=1 and kernel injection disabled (kernel injection was incompatible with this Transformers/Llama version). DeepSpeed auto-applied 8-bit quantization (quantize_bits=8), which accounts for both the reduced memory footprint and the latency improvement. On a single GPU its tensor-parallel benefit cannot be demonstrated.

Memory Footprint

Quantization's most significant resource benefit is memory reduction, measured directly during loading:

Configuration	Memory Footprint
Full precision (baseline)	4714 MB
4-bit quantized	1466 MB (~69% reduction)

Trade-off Analysis

Which method gave the best performance improvement?

4-bit quantization produced by far the largest gain among the directly comparable stages, taking mean latency from approximately 17.96 seconds to 0.59 seconds — roughly a 30x speedup — and raising throughput from under 1 token per second to about 30, while cutting memory footprint from 4714 MB to 1466 MB (about 69%). Its ROUGE-1 (0.142) is modestly below the baseline (0.191), a small quality cost for a very large efficiency gain. The mechanism is that headline generation is memory-bandwidth bound: most of the time is spent moving model weights between GPU memory and compute units. Representing weights in 4 bits rather than 16 dramatically reduces the volume of data moved per token, which is where the speedup comes from.

KV caching delivered the next most important improvement among the core stages, reducing latency from about 17.96 seconds to 4.52 seconds (a roughly 4x speedup). Without caching, the model recomputes attention over the entire growing sequence at every step; caching stores the key and value tensors from prior steps so each new token only attends once. This is the highest-value change relative to its near-zero implementation cost. Among the advanced stages, speculative decoding and DeepSpeed both reached the ~1.3–1.5 second range, but with important caveats: speculative decoding used a smaller gpt2 model (not comparable to the Llama rows), and DeepSpeed's speed came from auto-applied 8-bit quantization rather than parallelism on this single GPU.

Did any method significantly hurt quality?

None of the techniques catastrophically hurt quality, though there is more movement than an idealized analysis would predict. KV caching is lossless by construction (its ROUGE difference from baseline is numerical noise). Quantization held ROUGE-1 at 0.142 versus the baseline's 0.191 — a modest drop — showing the headline task tolerates 4-bit weights reasonably well. Pruning at 30% preserved ROUGE-1 (0.182), close to baseline. The ROUGE values should be read with the caveat that ROUGE measures surface word overlap, not meaning: an abstractive model can write a good headline using different words and still score modestly.

A note on the pruning implementation: the pruning applied here is unstructured (it zeros individual low-magnitude weights but leaves tensor shapes unchanged). On standard GPU hardware this yields no automatic speedup or memory savings, because the zeros are still stored and multiplied. This is borne out in the data: the pruned model's latency (4.78s) and memory (4714 MB) are essentially the same as KV caching alone (4.52s, 4714 MB), meaning pruning contributed no additional speed or memory benefit. Realizing pruning's performance benefit requires structured pruning or specialized sparse kernels. This is a meaningful finding rather than a failure: not every theoretical optimization translates to wall-clock gains on commodity hardware.

A note on model choice

The speculative decoding experiment surfaced an important point that sits underneath the whole project: the gpt2-based run scores far lower on ROUGE (0.093 versus ~0.16 for the Llama rows) not because of any optimization, but because base gpt2 is not instruction-tuned and therefore tends to continue the article rather than summarizing it. Inference optimizations

change speed and resource cost; they do not change what a model is fundamentally capable of. Model selection and instruction tuning determine quality, and no amount of inference optimization makes a base model good at a task it was never tuned for.

Distributed Inference: Parallelism and DeepSpeed

Because the available hardware was a single Tesla T4, true multi-GPU parallelism could not be exercised. Three approaches were investigated to characterize the distributed-inference dimension within that constraint: simulated pipeline parallelism, simulated tensor parallelism, and DeepSpeed Inference.

Pipeline parallelism partitions the model's decoder layers into sequential stages, each of which would live on a separate device in a real deployment. The simulation splits the 28 Llama layers into two stages and measures the per-device memory (total $\div$ 2) and end-to-end latency. Tensor parallelism instead shards each individual weight matrix across devices; the simulation verifies the shard-and-gather mechanics on a single linear layer (confirming the sharded output matches the unsharded reference) before measuring full-generation timing. In both cases, on one GPU the simulation cannot produce a genuine speedup — both halves of the split execute on the same device — so the value is demonstrating the partitioning mechanics and the per-device memory reduction these techniques provide, which is what enables running models too large for a single GPU.

DeepSpeed Inference (v0.17.4) was successfully initialized and run after resolving several version incompatibilities with the environment's Transformers/Llama stack (kernel injection had to be disabled, and inputs explicitly co-located with the engine's device). It completed a full 25-sample benchmark at 1.47 seconds mean latency and 34 tokens per second. Notably, DeepSpeed automatically applied 8-bit quantization during initialization, which explains both its reduced memory footprint (about 2472 MB) and its latency improvement over the fp16 baseline. With `tensor_parallel` set to a single device, DeepSpeed's core multi-GPU benefit could not be demonstrated; the measured gain reflects its automatic quantization rather than parallelism. The key takeaway is consistent across all three approaches: on a model that already fits comfortably on one GPU, distributed-inference techniques provide no latency benefit — their purpose is to enable larger models or to scale throughput across multiple devices, neither of which is the binding constraint for a 3B model on a T4.

Environment and Reproducibility

The benchmarks were run in the environment documented below. All timing used a warm-up run plus `torch.cuda.synchronize()` around the measured region to account for asynchronous GPU execution; throughput is total tokens divided by total elapsed time, and ROUGE is averaged over a fixed sample of 25 articles with an identical prompt across all stages.

Component	Value
GPU	NVIDIA Tesla T4 (Turing, compute 7.5, 15.64 GB GDDR6)
GPU driver	575.57.08
CUDA (PyTorch build)	12.8

Component	Value
cuDNN	91002
Python	3.10.14
PyTorch	2.10.0+cu128
Transformers	5.7.0
bitsandbytes	0.46.1 (4-bit NF4 quantization)
DeepSpeed	0.17.4 (distributed inference)
accelerate	1.10.0
datasets	2.19.1
rouge_score	installed (quality evaluation)
Primary model	Llama-3.2-3B (/voc/shared/models/llama/Llama-3.2-3B)
Spec. decoding draft / target	gpt2 / gpt2-medium (Llama-3.2-1B available as alt draft)
Dataset	News headline generation dataset
Evaluation sample size	N = 25 articles (identical prompt across stages)
Platform	Linux 5.15.0 (AWS x86_64), glibc 2.31

Recommendation

For deployment in a production environment at the news portal, 4-bit quantization is the recommended optimization strategy, ideally combined with KV caching (which any standard generation pipeline enables by default).

Performance: It provides the best latency and throughput of any technique tested, by a wide margin, which directly improves user-facing responsiveness and the number of requests a single GPU can serve.

Cost: The ~69% memory reduction means the model fits on smaller, cheaper GPUs and more model replicas fit per device, lowering serving cost per request.

Complexity: Quantization is straightforward to apply through the bitsandbytes integration in Transformers, requiring only a configuration change at load time rather than custom infrastructure.

Quality: The numerical perturbation from 4-bit quantization has a modest effect on output quality, an acceptable trade for the large efficiency gains in a headline generation use case.

Speculative decoding is a promising complementary technique for further latency reduction, but to be comparable here it should be re-run with a matched model family (a Llama-3.2-1B draft paired with the Llama-3.2-3B target) rather than gpt2, so it operates on the actual task model. Pipeline and tensor parallelism, and DeepSpeed's tensor-parallel mode, become relevant only when the model is too large for one device or when scaling throughput across multiple GPUs — neither of which is the binding constraint for a 3B model that already fits on a single T4. For this

specific workload, quantization plus KV caching is the pragmatic, well-supported production choice.